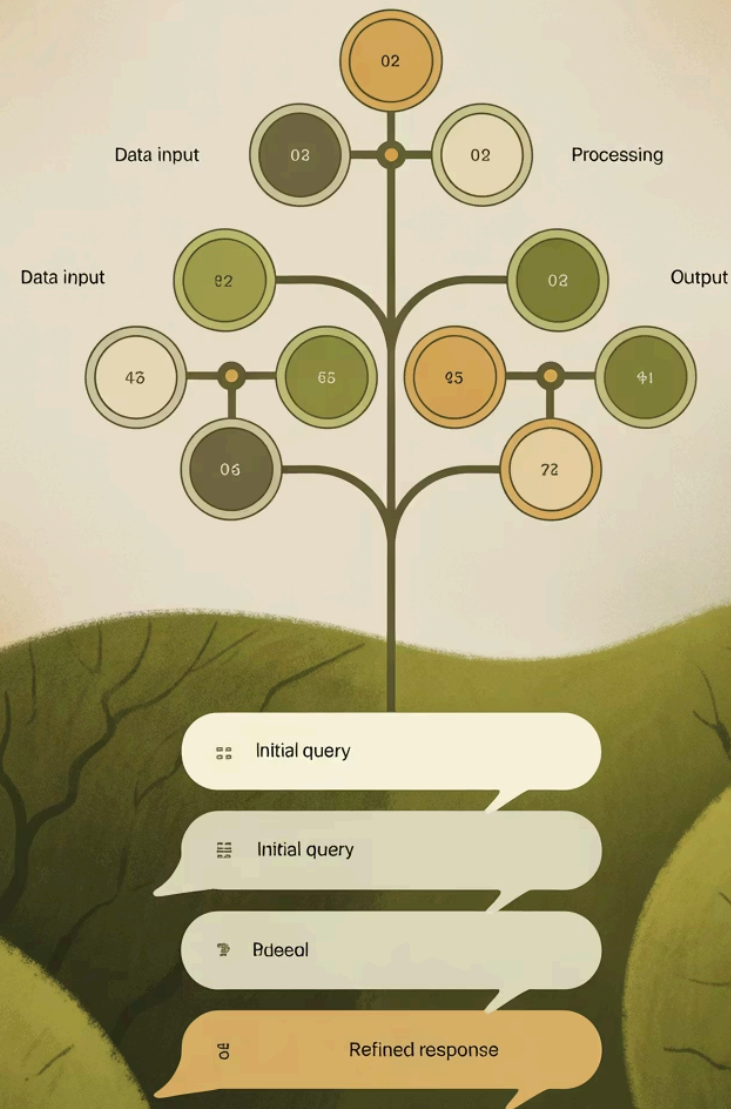


非常好的建議！這兩個改進方向將顯著提升 SalesRAG 系統的智能化程度

讓我詳細分析這兩個建議的可行性和實作方案。

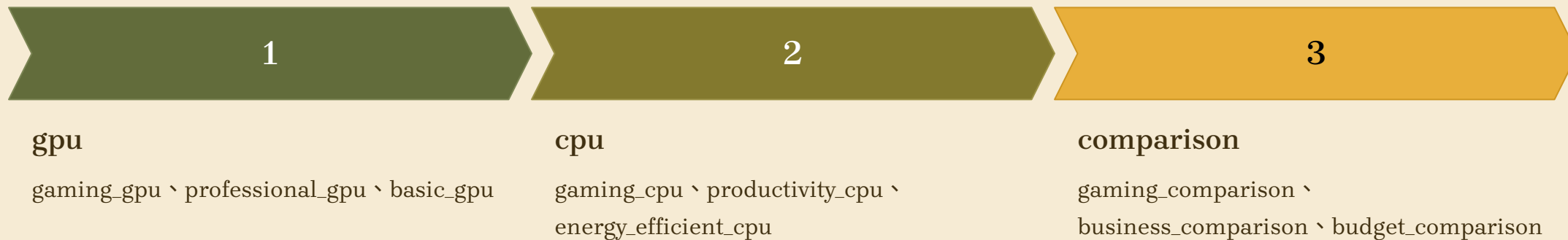


建議 1：更細緻的意圖分類系統

改進概念

將現有的 11 種基礎意圖進一步細分為更具體的使用場景意圖，例如：

現有意圖 » 細分意圖





建議的新結構

```
{ "intent_keywords": {  
  "gaming_performance": {  
    "keywords": ["遊戲", "gaming", "電競", "FPS", "幀率", "顯卡性能"],  
    "description": "遊戲性能相關查詢",  
    "parent_intent": "gpu",  
    "priority": "high",  
    "related_specs": ["gpu", "cpu", "memory", "display"]  
  },  
  "business_productivity": {  
    "keywords": ["辦公", "工作", "商務", "文書", "Excel", "PowerPoint"],  
    "description": "商務辦公相關查詢",  
    "parent_intent": "cpu",  
    "priority": "medium",  
    "related_specs": ["cpu", "memory", "battery", "portability"]  
  },  
  "content_creation": {  
    "keywords": ["影片編輯", "設計", "繪圖", "渲染", "Adobe", "創作"],  
    "description": "內容創作相關查詢",  
    "parent_intent": "gpu",  
    "priority": "high",  
    "related_specs": ["gpu", "cpu", "memory", "storage"]  
  }  
}}
```

優勢分析

1

精準推薦

系統能提供更貼合使用場景的建議

2

個人化體驗

根據用戶類型提供差異化回應

3

專業指導

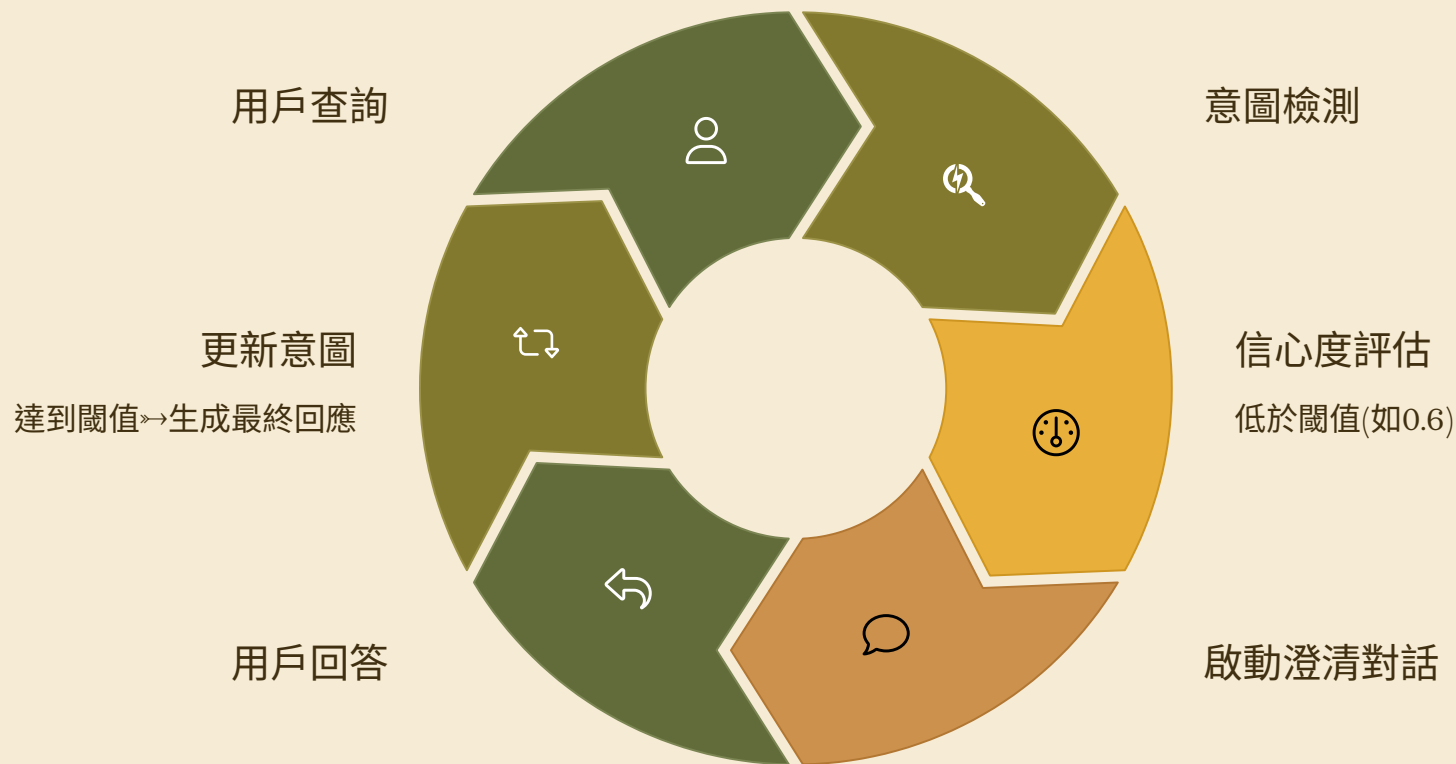
針對特定需求提供專業建議

建議 2：智能對話澄清系統

改進概念

當系統檢測到用戶意圖不明確（信心度 < 閾值）時，主動發起澄清對話，逐步收集用戶需求資訊。

對話澄清流程





澄清問題範例

情境 1：模糊的性能查詢

① 用戶："我想要一台性能好的筆電"

系統："為了更好地為您推薦，請問您主要用途是什麼？

- a. 🎮 遊戲娛樂
- b. 📁 辦公工作
- c. 🎨 設計創作
- d. 📖 學習研究"

情境 2：預算導向查詢

① 用戶："有什麼便宜的筆電推薦？"

系統："請問您的預算範圍大約是多少？

- a. 💰 經濟型（2-3萬）
- b. 🏠 中階型（3-5萬）
- c. 💎 高階型（5萬以上）"

技術實作方案

1. 細緻意圖分類實作

修改 query_keywords.json 結構

```
{ "intent_keywords": {  
  "基礎意圖": {  
    "sub_intents": {  
      "細分意圖": {  
        "keywords": [],  
        "description": "",  
        "scenarios": [],  
        "priority_specs": []  
      }  
    }  
  }  
}}
```

新增階層式意圖檢測方法

```
def detect_hierarchical_intent(self, query: str) -> dict:  
    # 1. 檢測基礎意圖  
    base_intents = self.detect_base_intents(query)  
  
    # 2. 在基礎意圖下檢測細分意圖  
    sub_intents = []  
    for base_intent in base_intents:  
        sub_intent = self.detect_sub_intent(query, base_intent)  
        if sub_intent:  
            sub_intents.append(sub_intent)  
  
    # 3. 計算組合信心度  
    return self.calculate_hierarchical_confidence(base_intents, sub_intents)
```

技術實作方案 (續)

2. 智能澄清對話實作

澄清對話管理器

```
class ClarificationManager:
    def __init__(self):
        self.clarification_templates = self._load_clarification_templates()
        self.conversation_state = {}

    def should_clarify(self, intent_result: dict) -> bool:
        """判斷是否需要澄清"""
        return intent_result["confidence_score"] < 0.6

    def generate_clarification_question(self, intent_result: dict) -> str:
        """生成澄清問題"""
        unclear_aspects = self._identify_unclear_aspects(intent_result)
        return self._create_clarification_prompt(unclear_aspects)

    def process_clarification_response(self, user_response: str, conversation_id: str) -> dict:
        """處理用戶的澄清回應"""
        # 更新對話狀態
        self._update_conversation_state(conversation_id, user_response)
        # 重新評估意圖
        updated_intent = self._reassess_intent(conversation_id)
        return updated_intent
```


澄清問題模板系統

```
CLARIFICATION_TEMPLATES = {  
  "usage_scenario": {  
    "question": "請問您主要的使用場景是？",  
    "options": [  
      {"id": "gaming", "label": "
```



實作效益預估

量化改進指標



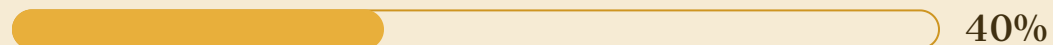
意圖識別準確率

預期從 70% 提升至 85%



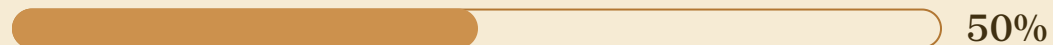
用戶滿意度

透過精準推薦提升 30%



對話完成率

減少無效查詢 40%



推薦相關性

提升 50%

技術架構優勢

模組化設計

澄清系統獨立運作，不影響現有功能

漸進式改進

可分階段實作，降低風險

數據驅動

收集對話數據持續優化澄清邏輯

用戶體驗

引導式對話提升使用便利性



進階功能展望

學習型澄清系統

- 模式學習: 從歷史對話中學習有效的澄清策略
- 個人化記憶: 記住用戶偏好，減少重複澄清
- 上下文理解: 基於對話歷史提供更智能的澄清

多模態澄清

- 圖片選擇: 透過產品圖片讓用戶選擇偏好
- 規格滑桿: 視覺化的規格選擇介面
- 使用場景模擬: 透過情境描述幫助用戶表達需求

這兩個建議的實作將讓 SalesRAG 從「被動回應」進化為「主動理解」的智能銷售助手，大幅提升用戶體驗和推薦準確性。